The Installation of ZeTa

Wolfgang Grieskamp

July 26, 2000, for version 1.5 of ZeTa

Abstract

This document describes how to install ZeTa 1.5, both binary and source distribution.

Contents

1 Requirements **1**

2 Installing the Binary Distribution **2**

2.1 Java Problems 3

2.2 Hand-crafting 3

3 Installing and Compiling the Source Distribution **3**

3.1 Getting the Make System to Work 3

3.2 Just Adding Adaptors 4

1 Requirements

You need to have a Java 1.2 or later on your system (e.g. jdk[1]). See section 2.1 for possible problems with the various Java implementations.

You should have XEmacs[2] (version 20 or later). XEmacs isn't strictly required for the the operation of ZeTa, but on its base a productive GUI is provided, and even for ZeTa's Swing based GUI, XEmacs is used for visiting source files.

If you are going to install XEmacs on your system for the first time, it is recommended that you enable the word-processing package AUC TeX for editing Z/LaTeX.

You need to have LaTeX2e installed in order to typeset documents. Binary kits for various architectures are found on the usual CTAN archives (for example, this[3]).

ZeTa is known to run under Linux (with glibc 2), Windows NT and Solaris (5.7). The execution machine ZAM, which is implemented in C++ and connected to ZeTa via a DLL, is pre-compiled exactly for these architectures. If you have a different architecture you may need to download the ZeTa sources and compile the ZAM yourself (see section 3). Whether Windows 98 or 2000 works with the NT DLL has not been tested, but there is a good chance.

[1] http://www.javasoft.com
[2] http://www.xemacs.org
[3] http://www.dante.de

2 Installing the Binary Distribution

The distribution is contained in a self-extracting class file, called `zetadist1_5.class`.

Under Windows NT, in order to let the ZeTa installer and XEmacs have the same notion of the home directory, you should ensure that the environment variable `HOME` is set to your needs (for example, `\WINNT\Profiles\USER`).

The installer will ask for locations of XEmacs and other client programs. It will make suggestions based on examining the PATH variable. Under a well configured Unix system, the required tools should be located automatically.

> *Remark*: Under NT, you may extend your PATH variable to the directories of the XEmacs and LaTeX distribution which contain the executables. Adding a user PATH variable turns out to be tricky if one is not already present ... select an existing user variable and then re-edit it. Editing the System PATH variable has no actual effect.

To start the actual installation, type

```
java zetadist1_5
```

where `zetadist1_5.class` needs to be found in your `CLASSPATH`, and `java` is the Java interpreter.

You are guided through the installation process by a graphical user interface. After finishing installation, a few things need still to be done:

- You may wish to put `$ZETABASE/bin` in your search path, where `$ZETABASE` is the installation directory
- Under NT, you may wish to drag `$ZETABASE\bin\runzeta.exe` to your desktop. This is a launcher for ZeTa which asks for the working directory where a ZeTa session shall start.
- If you have not selected the automatic patching of your `.emacs` file, you may want to include code as the following:

  ```
  (setq zeta-dir (expand-file-name "... fill in $ZETABASE .../"))
  (setq zeta-mode-default-on t)
  (setq zeta-vtex-default-on nil)
  (require 'zeta-site-init (concat zeta-dir "lib/emacs/zeta-site-init.el"))
  ```

  The following remarks:

  – the above code produces an error if not running under XEmacs. If you want to maintain a `.emacs` which should work as well under FSF Emacs, guard the code:

    ```
    (defvar running-xemacs (string-match "XEmacs\\|Lucid" emacs-version))
    ...
    (if (not running-xemacs)
        nil ;; do nothing
      ;; insert code as above
     )
    ```

  – the trailing slash of the `zeta-dir` value is required

  – if `zeta-mode-default-on` is `t`, then whenever you visit a file whose name ends with `.tex`, `.zed`, `.zeta`, `.esz`, or `.msz`, the zeta-mode will be enabled. You may want to set this variable to `nil`, if you do not intend to operate ZeTa from within XEmacs

  – if `zeta-style-hook-on` is `t`, then not only file endings, but also AucTeX style hooks will be used to activate the ZeTa. If you don't know what style hooks are, don't mind, and use the above setting.

2.1 Java Problems

- If you have a Blackdown JDK 1.2 on Linux, and you experience an exception of the kind

  ```
  An unexpected exception has occured:
  internal Java error

  java.io.IOException
          at zeta.tools.latex.ASCII_CharStream.FillBuff(Compiled Code)
          ...
  ```

  then you probably need to deactivate the JIT compiler by adding `"-Djava.compiler="` to the Java command line in `$ZETABASE/bin/zeta`. Its a bug in the compiler, not in ZeTa.

- Sun's Java 1.3 Beta Release for Linux does not seem to work at the moment. There are problems with the JNI. (Java 1.3 for Windows seems to operate fine, however.)

2.2 Hand-crafting

The automatic installation process creates files `$ZETABASE/etc/defaults.sh`, `$ZETABSE/etc/defaults.props` and `$ZETABSE/lib/zetarc`, which are relevant for hand-crafted customization of the ZeTa configuration.

3 Installing and Compiling the Source Distribution

Prior to installing the sources, you need to have successfully installed the binary distribution.

The sources are installed in the same directory as the binary distribution. Unpack `zetadistsrc.jar` in this directory, using the `jar` tool.

3.1 Getting the Make System to Work

- persistently set the environment variable `ZETABASE` to point to the installation directory of the ZeTa distribution
- unpack zetadistsrc.jar in `$ZETABASE`
- set execution permission for the added files `$ZETABASE/bin/{etmake,pc}`
- create a directory `$ZETABASE/classes` und unpack `$ZETABASE/lib/classes.jar` in this directory (the last one are the classes of the binary distribution)
- download and install JavaCC 1.n (http://www.metamata.com).
- edit the file `$ZETABASE/etc/defaults.sh` to your needs. In particular, let the variable `JAVACC` point to the starter for the `javacc` parser generator
- make a copy of the ZeTa starter, `$ZETABASE/bin/zeta`, to e.g. `dzeta`, such that in the copy `$ZETABASE/classes` is in the `CLASSPATH` instead of `$ZETABASE/lib/classes.jar`

You can now use `$ZETABASE/bin/etmake` for compilation of nodes. Launching `etmake` in `$ZETABASE/src` will compile the entire system; in an individual node, this node and the subnodes are compiled. Use `etmake install` in a node to install its objects and the objects of subnodes in the runtime environment of the ZeTa installation.

Note that many sources of ZeTa are written in Pizza[4] (the Pizza compiler is included in the distribution). However, the public interfaces are accessible from Java as well, since they only use class cases. There is a canonical representation of Pizza's class cases in Java. For instance:

```
class Expr {
  case Var(String name);
  case Apply(Expr fun, Expr arg);
}
```

This is represented in Java as:

```
class Expr {
  static class Var extends Expr {
    String name;
    Var(String name) { this.name = name; }
  }
  static class Apply extends Expr {
    Expr fun; Expr arg;
    Apply(Expr fun, Expr arg) { this.fun = fun; this.arg = arg; }
  }
}
```

For compiling `$ZETABASE/src/tools/nzap/zam` (the execution machine) you need a recent C++ compiler which supports the standard. It has been successfully compiled with gcc 2.95 (but not with the former egcs versions), and it has compiled with VC 6.0 (which, however, required some downgrades in template programming).

There are several cyclic dependencies between the nodes of the sources: this is the reason why we needed the binary distribution to compile the source distribution (the problem is that the Pizza compiler does not support `-dep` compilation). The make system provides a way for bootstrapping, which however has not been tested for this distribution: call `etmake boot` once in `$ZETABASE/src` and thereafter in `$ZETABASE/src/tools`.

3.2 Just Adding Adaptors

You do not need to compile the core system as described in the previous section to just add an adaptor. Compile your adaptor with your favorite Java-IDE against `$ZETABASE/lib/classes.jar` and plugin the beast using the command `plugin <qualified-classname>` in one of ZeTa's rc-files.

However, it may be useful to derive your adaptor from existing ones. Those may be of interest in the source distribution:

- `src/tools/latex` is a typical instance of an adaptor importing sources into the system
- `src/tools/zchecker/CheckerAdaptor` is a typical instance of an adaptor performing a transformation (that is mapping from one content into another)
- `src/tools/pp` is an instance of an adaptor which just produces output
- `src/tools/emacs/adaptor` is an instance of an adaptor which integrates an editor

If you write a new adaptor for the ZeTa system, please share it (contact `zeta@uebb.cs.tu-berlin.de` or `wg@cs.tu-berlin.de`). ZeTa currently most urgently misses adaptors for the integration of editors under Windows other than XEmacs, for other input formats of Z than LaTeX (e.g. ASCII markup) and for provers.

[4] http://www.cis.unisa.edu.au/ pizza